Document 03 — LLM Basics

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

02_apis_http_json

How to Read & Practice This Document

- **What:** how a language model actually works when it's answering you.
- **Why:** without this, every prompt you write is a guess, and every wrong answer feels like a random mystery instead of something you could have expected.
- **When:** before writing any prompt that matters — every decision about cost, length, and randomness comes back to this document.
- **How to practice:** 1. Read the material once, just to get the shape of it. 2. Do the **Basic** guessing exercises closed-book, then check yourself against the real tokenizer tool. 3. Do the **Intermediate/Real-world** exercises, and compare your guesses to what actually happens. 4. There's no coding in this document — the goal is a well-tuned gut feeling. Test it: guess first, then check. 5. Stuck on an idea, not code? Ask for **Hint 1** through **Hint 4** — the hint system works for ideas too, not just code. 6. Before moving on, explain out loud why a model making things up isn't a "bug" the same way a crash is. If you can't, you're not done.

Jump to: Core Concepts · Practice Exercises

The Story — what this document is actually building

Doc02 taught you that the network can fail in specific, predictable ways. This document is about a different kind of unpredictability — the model itself, and what actually happens inside a single call to it. There's no code to build here, but everything you learn becomes a gut feeling you'll use in every prompt you write from Doc04 onward.

First, the model never sees your words the way you do — it sees **tokens**, small chunks of text turned into numbers. Every limit and every price is measured in tokens, not words, so "how long is my prompt" is really "how many tokens is my prompt."

Second, everything you send and everything the model sends back shares one **context window** — a shared budget, not a one-way "how much can I paste in." And because the model has **no memory** of its own, every earlier turn of a conversation gets resent, and re-paid for, on every new call — which is also why a long conversation eventually risks the context window itself.

Third, the model doesn't just spit out one fixed answer — at each step it picks from a list of likely next tokens, and **temperature** controls how randomly it picks. Low temperature for tasks that need the same answer every time, higher temperature for tasks where variety is good.

Fourth, since input and output tokens are priced differently, and a long resent history multiplies both, cost is something you can actually estimate in advance, before you spend a cent.

Fifth, a model **making things up** isn't a bug you can patch — it's predicting "what sounds like a natural next word," with no built-in way to check that against the real world. Knowing this changes how you react to it (mitigate, don't expect to eliminate).

That's the story: tokens, context windows, memory, temperature, cost, and hallucination are one connected picture of what actually happens when you call a model — not five separate trivia facts. There's no Build Task here because this document's job is intuition, not code — Doc04 is where you start writing the calls this intuition will guide.

Core Concepts (read this first — everything you need is here)

Topics on this page: Tokens: what the model actually reads · Context window: a shared budget, not just "how much you can paste in" · No memory: there's nothing "remembered" inside the model · Randomness: `temperature` and `top_p` · Cost: input and output tokens are priced differently · Making things up: not a bug, just how the model works

Tokens: what the model actually reads

A token is not a word — it is a small chunk of text the model turns into a number: sometimes a whole word, sometimes a fragment, sometimes just punctuation. Think of a tokenizer like a vending machine that only accepts exact coins — your sentence gets broken into whatever exact "coins" the model's tokenizer knows, before the network ever sees a letter. As a rough guide, 1 token is about 4 characters of English, or roughly ¾ of a word, but that is a rule of thumb, not a fact, and it is the unit every limit and every price is measured in from here on.

How it really works

- Each model family ships its own tokenizer — a fixed list of text pieces learned before training, each with its own number. The same sentence gives a different token count on GPT-4o, Claude, and Llama.
- The character-count rule of thumb holds for plain English prose only. It breaks for non-English scripts (Urdu, Arabic and Chinese often need 2-3x more tokens for the same meaning, because

tokenizers are built mostly from English text), for code and JSON (symbols and indentation split into many tiny tokens), and for long numbers or IDs (digits often split token-by-token).

- Compact formatting saves real tokens on structured data — `json.dumps(row, separators=(",", ":"))` strips the spaces pretty-printed JSON pays for with nothing gained.
- The API always reports the real count after the call, in `response.usage` — the provider's own count, including small hidden extras like message formatting, and it is what you are actually billed on. Log it (same `logger.info` habit as Doc01) instead of trusting your own guess.
- When a limit really matters — a hard budget, a batch job over thousands of texts — count exactly with the model's own tokenizer library instead of guessing; the rule of thumb can be off by 2x or more.

```
# pip install tiktoken
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4o-mini")
print(len(enc.encode("My order #58213 has not arrived yet.")))
```

`tiktoken` only knows OpenAI tokenizers — other providers give a token-counting endpoint or a `usage` field instead.

Situation	What to do	Why
Quick guess while writing a prompt	tokens ≈ characters ÷ 4	Good enough for "small" vs. "huge"
A hard limit or budget in code	Count exactly with `tiktoken` (or the model's tokenizer)	Rule of thumb can be off by 2x or more
Non-English text (Urdu, Arabic, Chinese...)	Count, don't guess	Other scripts split into more, smaller tokens
Code, JSON, long numbers, IDs	Count, and trim extra spaces/fields	Symbols and digits split into many tiny tokens
Checking the real bill after a call	Read `response.usage`	The provider's own count — what you actually pay

Common mistakes:

- *Mistake:* using `len(text.split())` or `len(text)` as a stand-in for token count, then setting a hard limit on it. → *Symptom:* works fine on short English test text, then fails or costs far more on real input with code, numbers, or another language. → *Fix:* count with the model's real tokenizer wherever a limit matters, and log `usage` to check your own assumption.
- *Mistake:* setting token budgets using English test messages, then shipping to non-English users. → *Symptom:* real users' messages use 2-3x more tokens than planned, so costs and limits blow

past estimates. → *Fix:* test with real text in the languages you'll actually serve before setting any limit.

Where you'll meet it: in Doc04, every response has a `usage` field — this is what you log and pay for, with the same logger from Doc01. In Doc08, you split documents into chunks measured in tokens, and non-English or code-heavy documents need bigger chunk budgets than English prose. In Doc11 and Project 4, every agent's prompt has its own token count, added up for the cost of one full run. In Doc13, tokens per run is one of the first numbers you record.

Quick cheat sheet:

- Limits and prices are in tokens, not words.
- Rule of thumb: 1 token ≈ 4 English characters ≈ ¾ of a word — English prose only.
- Non-English text, code, and numbers use more tokens than the rule suggests.
- Count exactly with the model's own tokenizer when a limit really matters.
- The API's `usage` field is the real number — log it, don't estimate it.

Context window: a shared budget, not just "how much you can paste in"

The context window is the largest number of tokens a model can handle in one call — and the text you send in and the reply it gives back **share this same budget**, along with the whole conversation history you resend every turn. Think of it as one shared suitcase, not two separate ones: it's tempting to picture "how much can I paste in," but the real limit is system prompt + history + new message + the reply you expect, all packed into the same bag.

How it really works

- Many models also cap the reply alone, separately and smaller than the full window — a 128,000-token-window model may only write up to about 16,000 tokens in one reply. Check your model's docs, not just its window size.
- Going over the limit doesn't fail the same way everywhere: some tools give a clear error, others silently cut text off. The second is worse — your code looks like it worked while quietly losing information.
- If there isn't enough room left for the reply, the reply stops mid-sentence and the API reports `finish_reason == "length"` instead of `"stop"`.
- Fitting is not the same as being read well — models pay less attention to details buried in the middle of a very long input (the "lost in the middle" problem), covered again in Doc08.
- Every earlier turn of a conversation gets resent — and paid for again — on every new call, because the model has no memory of its own (next topic). A growing chat risks the window and the bill at the same time.
- In a multi-agent system (Doc11), this is a per-agent problem: deciding what goes into shared state is really a context-window decision for every agent that reads it.

Situation	What to do	Why
One short question, one short answer	Nothing special	Far below any real limit
A document bigger than the window	Don't paste it all — retrieve the relevant parts (RAG, Doc08)	It won't fit, and even a near-full window gives weaker answers
A chat that keeps growing	Keep the system prompt + recent turns; drop or summarize older ones	Old turns fill the window and cost money on every call
You need a long reply	Reserve room for it: input + expected reply ≤ window	Otherwise the reply is cut off halfway
Agents passing work to each other	Pass a short summary or structured result, not the full transcript	Each agent's window fills fast if everyone gets everything

Common mistakes:

- *Mistake:* checking only "does my input fit?" and forgetting to leave room for the reply. → *Symptom:* the call fails, or the answer is cut off halfway, and short tests hide it because they never fill the window. → *Fix:* budget input + expected reply together, and cap reply length under the room left over.
- *Mistake:* pasting a whole document because it technically fits. → *Symptom:* the model gives weaker, vaguer answers even though nothing errored. → *Fix:* retrieve only the relevant parts (Doc08's RAG) — fitting is not the same as being read well.

Where you'll meet it: Project 1 (SupportDesk) has a planned failure where a long conversation silently goes past the window. Doc04 has an exercise where you hit the limit on purpose. Doc08 exists partly because company documents don't fit. In Doc09 and Doc11, what goes into shared state is a context-window decision for every agent that reads it.

Quick cheat sheet:

- Window = system prompt + history + new message + reply, all together.
- Always leave room for the reply; many models also cap reply length separately.
- `finish_reason == "length"` means the reply was cut off.
- Too big to fit? Retrieve the relevant parts (RAG) — don't paste everything.
- Fitting is not the same as being used well: details in the middle get less attention.

No memory: there's nothing "remembered" inside the model

Every API call to a chat model stands completely alone — the model carries no memory of any earlier call, unless your own code sends that earlier conversation back to it. This is just how the API works: it takes in a list of messages and returns the next one, and nothing you say in a call changes

the model itself. Any "memory" a chatbot seems to have is not the model remembering — it is your code keeping a growing list and resending the right slice of it every time.

How it really works

- Some APIs can store the conversation for you server-side — OpenAI's Responses API links a new call to an earlier one with `previous_response_id`. This saves you from resending the list yourself, but the model still reads the whole history as input tokens on every call: the cost and the context-window limit are exactly the same as managing it yourself.
- A chatbot must append **both** the user message and the assistant's own reply to the list, every turn. Skip the assistant's reply and the model "forgets" what it just said, because that turn was never sent back.
- A single global `messages` list in a web server mixes every user's conversation together. The fix is one history per user or session id — the same isolation idea as not sharing state across requests.
- Cost grows as a conversation gets longer even when each new message is short, because you re-pay for the entire resent history every single turn — this connects straight into the Cost topic below.
- In a multi-agent system, "memory" becomes the shared state object: the Reviewer only knows what the Writer decided if that decision was written into the state it reads, not because agents can "see" each other's calls.

Situation	What to do	Why
A one-off task (summarize one email)	Send one call, no history	Nothing earlier is needed
A chatbot in a terminal	Keep one `messages` list; append user **and** assistant every turn	Skip either and the model "forgets" that part
A web API with many users at once	Keep a separate history per user/session id	One shared list mixes different users together
A conversation that gets very long	Trim or summarize old turns	Full history hits the window and costs more every turn
Remembering a user across days	Save facts to a database, load them into the next session's prompt	The API forgets everything the moment the call ends
Several agents on one task	Put what they must share into a shared state object	Each agent's call is memory-less too

Common mistakes:

- *Mistake:* appending only the user's message to the history, never the assistant's reply. → *Symptom:* the model "forgets" what it just told you and can't build on its own earlier answer. → *Fix:* append both the `user` and `assistant` message, every turn.

- *Mistake:* one global `messages` list shared across every user of a web server. → *Symptom:* works perfectly solo-testing; with two real users, one user's bot can mention another user's private details. → *Fix:* keep a separate history per session or user id — never one shared list.

Where you'll meet it: Doc04 and Project 1, where you build chat memory yourself with a growing `messages` list. Doc09, where a checkpointer saves state between steps. Doc12, where histories move from a Python list into a database so they survive a restart. Project 10, where memory lasts across sessions. In Doc11 and Project 4, "memory" becomes shared state between agents.

Quick cheat sheet:

- The model remembers nothing between calls — your code is the memory.
- Append both the user message and the assistant reply, every turn.
- One history per user/session — never one shared global list.
- Server-stored conversations still bill the full history as input tokens.
- In multi-agent systems, shared state is the memory between agents.

Randomness: `temperature` and `top_p`

At each step the model doesn't pick "the next word" directly — it works out a probability for every possible next token, then picks one based on those odds, over and over. `temperature` controls how sharp or spread out those odds are: near 0 it almost always takes the single most likely token; higher values give less-likely tokens a real chance, which feels more "creative" or random. `top_p` cuts a different way — it only considers the smallest group of tokens whose combined odds add up to `p` (so `top_p=0.9` keeps just the top tokens making up 90% of the probability), dropping the very unlikely options entirely instead of reweighting them.

How it really works

- `temperature=0` does not guarantee the exact same answer every single time — there is some randomness in the systems behind the API too — but it gets close enough to treat as "consistent."
- Change `temperature` **or** `top_p`, not both — OpenAI's own docs recommend this, because changing both at once makes the combined effect unpredictable.
- On OpenAI models, temperature ranges 0 to 2 with 1 as the default; some newer reasoning models don't let you set it at all and return an error if you try.
- `temperature=0` makes an answer consistent, not correct — a wrong answer can repeat identically every run, which can hide the problem because every test run agrees with every other one.
- Different agents in one system commonly run different settings: a router or supervisor at 0 for predictable routing, a writer at 0.7-1.0 for natural variety in the same run.

Situation	What to do	Why
Pull fields into JSON, or sort into categories	`temperature=0`	Same input should give the same output; easy to test
A router or supervisor choosing the next step	`temperature=0`	Routing that changes randomly is very hard to debug
Normal chat answers	Leave the default	A little variety sounds natural
Brainstorming names, slogans, story ideas	`temperature` around 0.8-1.2	You *want* different answers each time
Several different drafts to choose from	Higher temperature + several calls	At 0 all drafts come out nearly the same
Above ~1.5	Rarely useful	Text starts to become strange or broken

Common mistakes:

- *Mistake:* thinking `temperature=0` makes an answer correct. → *Symptom:* a wrong answer repeats identically every run, and passes every test because the runs agree with each other, not with the truth. → *Fix:* verify against a real source, not against repeatability.
- *Mistake:* raising temperature hoping for a "smarter" answer. → *Symptom:* text gets more varied or stranger, not more accurate. → *Fix:* use temperature only for variety; fix accuracy with better grounding (RAG, tools) instead.

Where you'll meet it: in Doc04 as the `temperature=` argument. Project 1's Triage agent uses `temperature=0`, because a ticket must get the same category every time. In Doc11 and Project 4, the Supervisor and Reviewer run at 0 to be predictable, while the Writer runs higher for natural text. In Doc13, you learn why tests on model output can still be "flaky" even at 0.

Quick cheat sheet:

- Temperature changes how randomly the next token is picked, not how smart the model is.
- `temperature=0` for extraction, classification, routing, and anything you test.
- Higher (around 0.8-1.2) for brainstorming and creative writing.
- Change `temperature` or `top_p`, not both.
- 0 means consistent, not correct — and not perfectly identical every time.

Cost: input and output tokens are priced differently

API pricing charges a rate per input token, and a separate, usually higher, rate per output token — generating new text costs more than reading existing text — and prices are usually written per 1 million tokens. Because a resent conversation history counts as input tokens on every call, and a

system prompt sent thousands of times adds up even at a cheap per-token rate, cost is something you can estimate ahead of time, not a surprise on the invoice.

How it really works

- Rough formula: (input tokens × input rate) + (output tokens × output rate), added up across every call in a session — and in a chat, "input tokens" means the *entire* resent history, not just the newest message (this is the No-memory topic above, showing up as a bill).
- Reasoning models "think" in extra tokens you usually don't see in the reply — those are still billed as output tokens.
- A retry after a timeout (Doc02) is a brand-new call — you pay for it again, on top of the one that failed.
- Prompt caching (covered again in Doc12) bills input the provider has seen very recently at a lower rate — the fix for a long, repeated system prompt.
- A multi-agent pipeline multiplies calls fast: every agent is its own paid call, and a review loop that runs 3 times reruns the writer and reviewer 3 times each.
- Price constants belong in a config object, loaded once — the same `.env` / `config.py` pattern from Doc01 — so a provider's price change is a config edit, not a code edit.

Example prices used in this document (made-up but realistic, only for practice — always check the provider's current pricing page):

Example model	Input (per 1M tokens)	Output (per 1M tokens)	Good for
"Small model"	$0.15	$0.60	Classification, routing, short answers
"Large model"	$2.50	$10.00	Hard reasoning, long careful writing

Situation	What to do	Why
A long system prompt sent every call	Keep it short; use prompt caching where available	You pay for it thousands of times
A long chat history resent every turn	Trim or summarize old turns	Input grows every turn, faster than the conversation itself
Long replies	Ask for short answers; set a max reply length	Output tokens cost more than input tokens
A simple task on a large model	Use a small model for easy steps	The same job can be 10-20x cheaper
A multi-agent pipeline	Add up every agent's calls; cap review rounds	Every agent is a separate paid call; loops multiply them

Common mistakes:

- *Mistake:* estimating cost from only the newest message, ignoring the resent history and the higher output rate. → *Symptom:* the estimate looks tiny; the real bill, once a conversation runs long, is many times bigger. → *Fix:* multiply against the *whole* resent history each turn, and log `usage` per call instead of estimating after the fact.
- *Mistake:* running every pipeline step on the largest model "to be safe." → *Symptom:* routing and classification cost 10-20x more than needed, with no accuracy gain. → *Fix:* small model for easy steps (routing, classification), large model only where it earns its cost.

Where you'll meet it: the Intermediate exercise below (cost of a 2,000-input/500-output call). Doc04, where you read `response.usage` after each call and log it. Doc12, where caching and model choice cut real bills. Doc13, where cost per run is logged and watched. In Doc11 and Project 4/5, cost is often the reason to use fewer agents or smaller models for some of them.

Quick cheat sheet:

- Cost = input tokens × input price + output tokens × output price, per call, added up.
- Output tokens usually cost more than input tokens.
- In a chat, input includes the whole resent history — it grows every turn.
- Use small models for easy steps; cap reply length and review loops.
- Always log `usage` per call so you know the real number, not a guess.

Making things up: not a bug, just how the model works

A language model writes text by repeatedly picking a likely next token based on everything before it — it has no built-in way to check "is this actually true" against the real world, only "does this sound like a natural continuation." This is usually called **hallucination**, and a confident, fluent, wrong answer is produced by the exact same process as a confident, fluent, correct one — there is no internal flag telling them apart, so you can't just patch it away.

How it really works

- Risk rises for a topic with little training data, for very precise facts (exact dates, numbers, quotes, links, sources), for anything past the model's training cutoff, for private data it never saw (your company's policy, your customers' orders), and whenever it is asked to fill a gap instead of being allowed to say "I don't know."
- Grounding (RAG, Doc08) gives the model real retrieved text to work from instead of a trained-in guess; tools and databases (Doc06) give it exact facts instead of a memory of facts. Neither *removes* the underlying cause — both make it show up less, and easier to catch.
- Asking the model "are you sure?" is not a check — the "yes," or the apology that changes a correct answer to a wrong one, comes from the same next-token process as the original answer, with no source behind either one.

- A made-up fact from one agent becomes "truth" to the next agent that reads it, unless a reviewer step checks claims against a real source before passing them on.
- Anything the model writes that looks like an ID, a citation, or a fact needs validating against your own real data before you trust it — an invented order ID is exactly as fluent and confident as a real one.

Situation	What to do	Why
Questions about your own private data	Give the model the real text (RAG); allow "I don't know"	The model never saw this data, so it can only guess
Exact numbers, dates, links, citations	Get them from a tool or database; check in code	Precise details are where fluent guesses are most often wrong
Summarizing text you gave it	Lower risk, but ask it to stay close to the source; spot-check	It can still add a "natural-sounding" detail that isn't there
Creative writing	Usually fine	Making things up is the goal
One agent's output feeds another agent	Add a reviewer step; check structured fields with Pydantic	A made-up fact from agent 1 looks like truth to agent 2

Common mistakes:

- *Mistake:* treating "ask the model if it's sure" as a verification step. → *Symptom:* the model says "yes" to a wrong answer, or apologizes and changes a correct one to a wrong one — both from the same process. → *Fix:* check against a real source (document, database, tool result), never against the model's own opinion of itself.
- *Mistake:* letting the model fill a gap instead of allowing "I don't know." → *Symptom:* a fluent, specific, wrong answer that reads exactly as confident as a correct one. → *Fix:* prompt it explicitly to say "I don't know" when the source doesn't cover the question, and treat that as a good answer, not a failure.

Where you'll meet it: the Failure exercise below, where you make it happen on purpose. Doc06, where the model can also invent a tool argument (like an order ID) that you must validate. Doc08, where RAG reduces this but doesn't remove it. Doc13, where you measure how often answers are not backed by a source. In Doc11 and Project 4, a reviewer/checker step is one of the main defenses — in a chain of agents, one made-up fact can travel all the way to the final output.

Quick cheat sheet:

- The model predicts natural-sounding text; it does not check facts.
- Wrong and right answers look equally confident.
- Highest risk: exact facts, links, citations, recent events, your private data.
- Ground answers in real text (RAG) and real data (tools); allow "I don't know."
- In multi-agent systems, add a reviewer that checks claims against sources.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- OpenAI Tokenizer (interactive tool) — paste in text, see it split into tokens.
- tiktoken (OpenAI's tokenizer library) — count tokens in code, not just in the browser tool.
- OpenAI — Prompt engineering guide — practical guidance straight from OpenAI.
- Anthropic — Building Effective Agents — not tied to one company, and very good on what LLMs are good and bad at. Worth reading again after Doc11.

Practice Exercises (no coding this document — guessing and estimating)

Why no code here: this document is about building correct intuition (tokens, cost, context limits), not a coding skill on its own — the code that uses this understanding starts in Doc04. If you want to run the token-counting exercise below in real code instead of the browser tool, `pip install tiktoken` and use it directly — that's the one optional script worth writing here.

Where your notes live: all of it under `03_llm_fundamentals/practice/` (`mkdir -p practice`) — not scripts, since there's no required code, but a written record of your guess vs. the real answer for each exercise. Writing the guess down *before* checking is what actually builds the intuition; skipping straight to the answer doesn't.

The full file layout, all exercises:

```
practice/
├── token_count_guessing_notes.md       Basic
├── temperature_cost_notes.md           Intermediate
├── context_window_capacity_notes.md    Real-world
├── token_rule_exceptions_notes.md      Edge cases
└── induced_hallucination_notes.md      Failure
```

Why each notes file exists:

- `token_count_guessing_notes.md` — the gut-feeling for tokens every cost/context-limit decision from here on depends on.
- `temperature_cost_notes.md` — proves temperature's effect with real output, and turns per-token pricing into a number you trust.
- `context_window_capacity_notes.md` — the intuition behind Project 1's "conversation silently exceeds context" failure, before it happens to you.

- `token_rule_exceptions_notes.md` — catches you before a multilingual or code-heavy feature quietly costs way more than estimated.
- `induced_hallucination_notes.md` — turns "the model hallucinated" from a mysterious event into an expected failure mode you can explain.

(Optional, not required: if you'd rather check token counts in real code than the browser tool, `pip install tiktoken` and call it directly — no fixed filename, your own throwaway script.)

Jump to an exercise: Basic · Intermediate · Real-world · Edge cases · Failure

Basic — guess token counts, then check

- **What:** guess the token count of 5 sentences of different lengths, then check with the tokenizer tool.
- **Why:** you need your own gut feeling for tokens before Doc04, because every cost and context-limit decision from here on depends on it.
- **When you'll hit this for real:** every time you write a system prompt and wonder "is this going to be expensive," before you've written a single line of code to check.
- **How to practice it:** open platform.openai.com/tokenizer, write down your guess for each sentence *before* pasting it in, then compare.
- **How to code it:** for each sentence, guess with words × 1.3 (and characters ÷ 4), write the guess down, then paste the sentence into OpenAI's tokenizer page and note the real count and how far off you were.
- **Save as:** `practice/token_count_guessing_notes.md` — your 5 guesses, the real counts, and how far off you were.
- **Used later by:** the token-budget gut feeling built here is exactly what the Intermediate exercise's cost math depends on, what the Real-world exercise scales up to a whole conversation, and what Doc08's chunk-size decisions and Doc04's context-limit exercise assume you already have.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — temperature and cost, side by side

- **What:** run one prompt at `temperature=0` and `temperature=1`, three times each, and separately work out the cost of a 2,000-input/500-output-token call.
- **Why:** seeing the actual variation (or lack of it) at each temperature setting is more convincing than reading about it — and cost math you've done by hand once, you'll estimate correctly forever after.
- **When you'll hit this for real:** choosing a temperature setting for a real feature (Project 1's Triage agent needs `temperature=0`; a creative-writing feature wouldn't).
- **How to practice it:** once you have Doc04's client, loop 3 calls at each temperature and print all 6 answers side by side. For cost: look up OpenAI's current per-token prices and multiply by hand.

- **How to code it:** send the same prompt 3 times at `temperature=0` and 3 times at `temperature=1` (in the OpenAI Playground, or a few lines with Doc04's `client.chat.completions.create(..., temperature=...)` once you've done it), and paste the 6 answers side by side. Then cost = input tokens ÷ 1,000,000 × the input price + output tokens ÷ 1,000,000 × the output price.
- **Save as:** `practice/temperature_cost_notes.md` — the 6 answers side by side, and your cost calculation.
- **Builds on:** the Basic exercise's token-guessing — the cost half of this exercise is that same guess, converted to dollars with the formula from the Cost topic above.
- **Used later by:** Doc04's own temperature choices for its exercises reuse this same reasoning — pick `temperature=0` where the answer must repeat, higher where it shouldn't.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — how long can this conversation actually go?

- **What:** guess the largest number of conversation turns that fit in a given context window, for an average message length you choose.
- **Why:** this is the exact question behind Project 1's "conversation silently exceeds the context window" failure — you're building the intuition that predicts it before it happens.
- **When you'll hit this for real:** designing any chat feature — you need to know roughly when to start trimming history, before a user's long conversation breaks silently.
- **How to practice it:** pick an average message length (e.g. 50 tokens), pick a context window size (e.g. 128k), and do the division — then sanity-check against a real long conversation if you have one.
- **How to code it:** pick an average message length, turn it into tokens (characters ÷ 4), double it for one question plus one answer, then divide the context window — minus the system prompt and room for the reply — by that number.
- **Save as:** `practice/context_window_capacity_notes.md` — your numbers and the division.
- **Builds on:** the Basic exercise's per-message token estimate — same arithmetic, scaled up to a whole conversation's worth of turns.
- **Used later by:** this is exactly the intuition behind Doc04's edge-case exercise on what happens when you go over the limit, and behind Project 1's planned context-window failure.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — where the ¾-word rule breaks

- **What:** guess the token count of a non-English prompt (or one full of code), before checking.
- **Why:** the "1 token ≈ ¾ of a word" rule of thumb quietly stops working outside plain English — you need to know that *before* it costs you real money on a multilingual or code-heavy feature.
- **When you'll hit this for real:** any project with non-English users, or one where you're sending code (not prose) to the model — both tokenize far less efficiently than English text.

- **How to practice it:** take one prompt in Urdu (or another non-English language) and one snippet of Python code, guess both token counts, then check both against the tokenizer tool. Note how far off you were, for each.
- **How to code it:** guess the tokens with the characters ÷ 4 rule, check the real count in the tokenizer, and compare the gap with the English sentences from the Basic exercise.
- **Save as:** `practice/token_rule_exceptions_notes.md` — both guesses, both real counts, and the gap.
- **Builds on:** the Basic exercise's tokenizer-checking habit, now applied where the rule of thumb is known to break.
- **Used later by:** Doc08's chunk-size math needs this same awareness for non-English or code-heavy source documents, or chunks come out far larger than planned.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — make it confidently wrong, on purpose

- **What:** write a prompt you're confident will make the model state something false, confidently.
- **Why:** doing this once, deliberately, is what turns "the model hallucinated" from a mysterious, alarming event into an expected, structural failure mode you can explain to someone else.
- **When you'll hit this for real:** the first time a user reports a wrong answer from your app, and you need to explain *why* it happened, not just apologize.
- **How to practice it:** ask about a very specific, obscure fact (an exact statistic, a niche detail) the model likely wasn't trained on well. Write one paragraph explaining, in your own words, why it happened — without using the word "mistake."
- **How to code it:** ask about something that sounds real but doesn't exist — a made-up research paper, a library function that was never written — save the confident answer, check the facts yourself, and explain in one paragraph why the model filled the gap instead of saying it didn't know.
- **Save as:** `practice/induced_hallucination_notes.md` — the prompt, the false answer, and your one-paragraph explanation.
- **Used later by:** Doc08's evals and Doc13's "answers not backed by a source" checks rely on you being able to recognize this failure mode by feel, before you have code that measures it for you.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Expected Behavior / What You Should Be Able to Predict

- Before running a prompt, you should be able to roughly guess: how many tokens it uses, roughly what it will cost, and whether it risks going over the context limit.
- You should be able to guess, without running it, whether raising the temperature will actually change a given prompt's answer much (some prompts barely change no matter the temperature; others change a lot).

Break-It / Debug Preview

- A prompt that quietly goes over the context limit (what actually happens — does it get cut off? does it error? which part gets cut?).
- A `temperature=1` call used somewhere that needs the same answer every time.
- Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- Why LLMs have no memory of their own · what a context window actually limits · why cost isn't simply "how much I asked for" · temperature vs. top_p · why making things up can't just be "fixed" like a bug.

Move On When

You can guess, before running it, roughly how a prompt will behave and roughly what it will cost. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full explanation only if you say **"Show me the solution."**